COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Full Stack Web Development - Class Test Report

Course Code: BTCS303T | Duration: 1 Hour | Maximum Marks: 20

Name: Mohd Raza	**CU ID:** cu24250044
Roll No: 35	**Section:** B

Q1. Node.js - The Event Loop

Question: *Predict the console output order and justify in 2-3 lines.*

Output:

```
A
D
C
B
```

Justification: Synchronous code runs first on the call stack (A, then D). Once the stack is empty, Node drains the microtask queue, which holds promise callbacks, so C prints. Only then does the event loop reach the timers phase and run the setTimeout callback (B), even though its delay is 0.

Q2. Node.js - Modules

Question: *Create calc.js exporting add and subtract; import them in app.js and log add(4, 5).*

calc.js

```
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }

module.exports = { add, subtract };
```

app.js

```
const { add, subtract } = require('./calc');

console.log(add(4, 5)); // 9
```

Q3. Express.js - Middleware & Routing

Question: *Write requireAuth middleware (x-api-key = secret123, else 401) and apply it only to POST /api/books.*

```
const express = require('express');
const app = express();
app.use(express.json());

// (a) custom middleware
function requireAuth(req, res, next) {
  if (req.header('x-api-key') !== 'secret123') {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next();
}

// (b) applied only to POST
app.get('/api/books', (req, res) => res.json([]));        // public
app.post('/api/books', requireAuth, (req, res) => {       // protected
  res.status(201).json({ message: 'Book created' });
});

app.listen(5000);
```

Q4. Express.js - Building a REST API

Question: *Implement GET, POST and DELETE for a Book resource using an in-memory array and correct status codes.*

```
const express = require('express');
const app = express();
app.use(express.json());

let books = [];
let nextId = 1;

// GET all
app.get('/api/books', (req, res) => {
  res.status(200).json(books);
});

// POST create
app.post('/api/books', (req, res) => {
  const { title, author } = req.body;
  if (!title || !author) {
    return res.status(400).json({ error: 'title and author are required' });
  }
  const book = { id: nextId++, title, author };
  books.push(book);
  res.status(201).json(book);
});

// DELETE by id
app.delete('/api/books/:id', (req, res) => {
  const index = books.findIndex(b => b.id === Number(req.params.id));
  if (index === -1) {
    return res.status(404).json({ error: 'Book not found' });
  }
  books.splice(index, 1);
  res.status(204).send();
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

Q5. Express.js - Debugging & Error Handling

Question: *Identify the bug in GET /api/books/:id and rewrite correctly.*

Bug: `books.find()` returns `undefined` when no book matches, so `book.title` throws a TypeError. Nothing handles it, so the request fails instead of returning a clean JSON error.

Fixed code:

```
app.get('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === Number(req.params.id));

  if (!book) {
    return res.status(404).json({ error: 'Book not found' });
  }
  res.status(200).json(book);
});
```

The fix checks for the missing book and returns a 404 JSON error. It also uses strict comparison with Number() instead of ==.

Q6. APIs - Consuming a REST API in React

Question: *Write BookList using fetch() in useEffect, handling loading, error and success states.*

```
import { useState, useEffect } from 'react';

function BookList() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('http://localhost:5000/api/books')
      .then(res => {
        if (!res.ok) throw new Error(`Request failed (${res.status})`);
        return res.json();
      })
      .then(data => setBooks(data))
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading books...</p>;
  if (error) return <p>Something went wrong: {error}</p>;

  return (
    <ul>
      {books.map(b => (
        <li key={b.id}>{b.title} by {b.author}</li>
      ))}
    </ul>
  );
}

export default BookList;
```

Q7. APIs - CORS

Question: *Explain in 2 lines why the browser blocks the request, and write the Express fix.*

Why it happens: Browsers enforce the same-origin policy. localhost:3000 and localhost:5000 are different origins (the port differs), so the browser blocks the response unless the server sends Access-Control-Allow-Origin headers. Postman is not a browser and does not enforce CORS, which is why the request works there.

Fix:

```
const cors = require('cors');   // npm install cors

app.use(cors({ origin: 'http://localhost:3000' }));
// for quick local development, app.use(cors()) also works
```

Place this before the route definitions.

- End of Report -